Practice Exam: Python Programming and Version Control

Course: PRA2931 - Python Programming
Duration: 2 hours
Total Points: 100
Materials: Closed book, no internet access

Instructions

- Read all questions carefully before starting
- Answer all questions
- For code reading: trace through mentally and write the output
- For debugging: identify what's wrong and explain the fix
- Explain your thought process for partial credit
- Manage your time: ~12 minutes per 10 points

Point Distribution:

- Part 1 (Git/Version Control): 15 points
- Part 2 (OOP Fundamentals): 25 points
- Part 3 (Advanced OOP): 30 points
- Part 4 (Coding Exercises): 30 points

Part 1: Git and Version Control

1. Explain the difference between `git commit` and `git push`. When would you use each command, and what happens to your code in each case? (4 points)

2. You're working on a feature branch called `feature-analysis`. Put the Git commands in the correct order to create and switch to the feature branch, make changes and commit them and merge your feature back into `main`. (5 points)

Command	Order
git add.	
git checkout main	
git checkout -b feature-analysis	
git commit -m “Add analysis feature”	
git merge feature analysis	

3. What is a merge conflict? Give an example of when it occurs and describe the steps to resolve it. (6 points)

Part 2: OOP Fundamentals (25 points)

1. Take a look at this code. Trace trough it carefully.

```python
class Counter:
    count = 0

    def __init__(self, name):
        self.name = name
        Counter.count += 1

    def increment(self):
        Counter.count += 1

c1 = Counter("First")
c2 = Counter("Second")
c1.increment()
print(f"Count: {Counter.count}")
print(f"c1.count: {c1.count}")
print(f"c2.count: {c2.count}")
```

a) What is the output of this code? Write the complete output.

b) Explain why `c1.count` and `c2.count` have the same value.

2. This code has a bug:

```python
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return "Some sound"

class Dog(Animal):
    def __init__(self, name, breed):
        self.breed = breed

    def speak(self):
        return "Woof!"

dog = Dog("Rex", "Labrador")
print(dog.name)
print(dog.speak())
```

a) What error occurs?

b) Why does this error occur?

c) How would you fix it?

3. Fill in the blanks in the code.

```python
class Book:
    def __init__(self, title, author, pages):
        self.title = title
        self.author = author
        self.pages = pages

    def summary(self):
        # Return: "Title by Author (X pages)"
        return ________________________________

class TextBook(Book):
    def __init__(self, title, author, pages, subject):
        # Call parent __init__
        ______________________________________
        self.subject = subject

    def summary(self):
        # Return: "Title by Author - Subject textbook (X pages)"
        parent_summary = _________________________
        return ______________________________________

# Test
book = Book("1984", "Orwell", 328)
print(book.summary())  # Should print: "1984 by Orwell (328 pages)"

textbook = TextBook("Biology", "Smith", 450, "Science")
print(textbook.summary())  # Should print: "Biology by Smith - Science textbook (450 pages)"
```

Part 3: Advanced OOP

1. Look at the code. What is the output? If there is an error, identify where and why.

```
class Temperature:
    def __init__(self, celsius):
        self.celsius = celsius

    @property
    def celsius(self):
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("Below absolute zero!")
        self._celsius = value

    @property
    def fahrenheit(self):
        return self.celsius * 9/5 + 32

# Test cases:
t1 = Temperature(25)
print(f"C: {t1.celsius}, F: {t1.fahrenheit}")

t1.celsius = 100
print(f"C: {t1.celsius}, F: {t1.fahrenheit}")

t2 = Temperature(-300)
print(f"C: {t2.celsius}")
```

2. Magic methods – what is missing?

This Vector class is incomplete. Identify what is missing in each magic method and explain what would go wrong.

```python
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return f"Vector({self.x}, {self.y})"

    def __add__(self, other):
        # Missing: return statement
        Vector(self.x + other.x, self.y + other.y)

    def __eq__(self, other):
        # Missing: comparison logic
        pass

v1 = Vector(1, 2)
v2 = Vector(3, 4)
print(v1)                 # What prints?
v3 = v1 + v2             # What happens?
print(v3)                # What prints?
print(v1 == Vector(1, 2))  # What prints?
```

a) What does ‘print(v1)’ output?

b) What happens with ‘v3 = v1 + v2’?

c) What does ‘print(v1 == Vector(1,2))’ output and why?

d) Fix the ‘__add__’ and ‘__eq__’ methods.

3. Context Managers

What is the output of this code?

```python
class Logger:
    def __init__(self, name):
        self.name = name

    def __enter__(self):
        print(f"Start logging: {self.name}")
        return self

    def __exit__(self, exc_type, exc_value, exc_traceback):
        if exc_type is None:
            print(f"End logging: {self.name}")
        else:
            print(f"Error in {self.name}: {exc_type.
            __name__}")
        return False

# Test 1:
with Logger("Test1"):
    print("Processing data...")

print("---")

# Test 2:
try:
    with Logger("Test2"):
        print("Processing...")
        raise ValueError("Something broke")
except ValueError:
    print("Caught error")
```

Write down the complete output in order:

Part 4: Coding exercise – Implement a class

Create a ‘Student’ class:

Requirements:

- Attributes: ‘name’ (str), ‘student_id’ (str), ‘grades’ (list, initially empty)
- Method ‘add_grade(grade)’: adds grade to list (validate 0-100)
- Method ‘ average()’: returns average grade (0 if no grades)
- Property ‘status’ (read-only): Returns “pass” if average >= 60, else “Fail”

Your code:

Class Student: